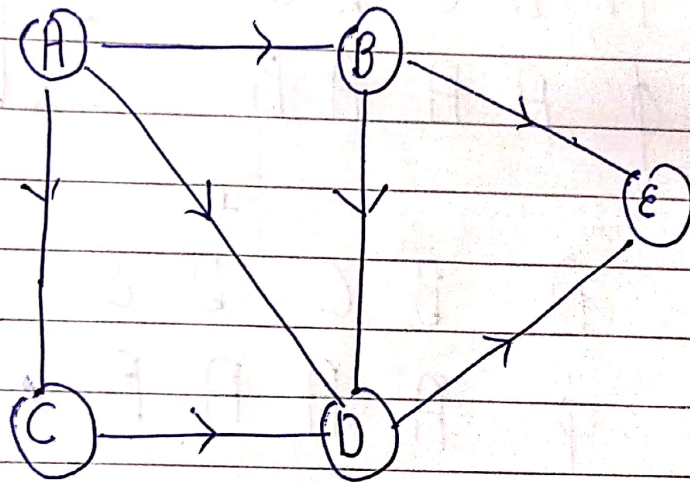


Graph Traversal

Breadth First Search - level by level traversal
- Queue is used



Vertex	Adjacency list
A	B, C, D
B	D, E
C	D
D	E
E	-

- Status array
- 1 - Initialize all node
 - 2 - Put the node in queue
 - 3 - Remove the node in queue

Status array

A	B	C	D	E
1	1	1	1	1

A	B	C	D	E
2	1	1	1	1

Step 1

Queue A
Orig ϕ

Status array

~~Step 2~~

~~Queue A Remove A~~

Step 2

①	A	^{f, x} B	C	^x D
Orig	φ	A	A	A

status array

A	B	C	D	E
3	2	2	2	1

Step 3

①	A	B	^f C	D	^x E
Orig	φ	A	A	A	B

status array

A	B	C	D	E
3	3	2	2	2

Step 3

①	A	B	C	^f D	E
Orig	φ	A	A	A	B

status array

A	B	C	D	E
3	3	3	2	2

Step 4

①	A	B	C	D	^f E
Orig	φ	A	A	A	B

status array

A	B	C	D	E
3	3	3	3	2

Step 5

①	A	B	C	D	E
Orig	φ	A	A	A	B

status array

A	B	C	D	E
3	3	3	3	3

Time Complexity of BFS

Using Adjacency matrix = $O(V^2)$

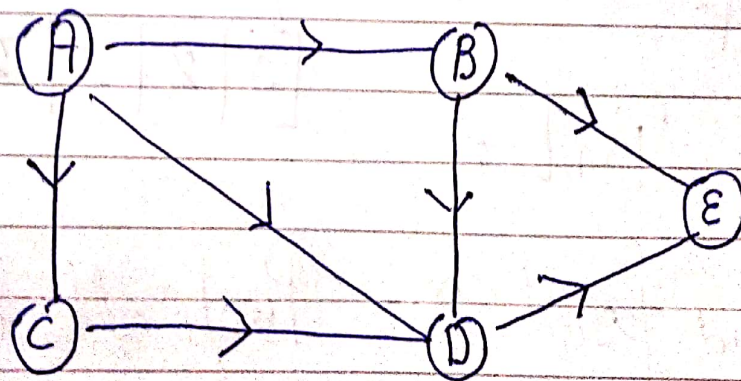
Using Adjacency list = $O(V+E)$

Space Complexity of ~~BFS~~ BFS

Using Adjacency matrix = $O(V^2)$

" Adjacency list = $O(V+E)$

DFS traversal



Adjacency list	
A	B, D, C
B	E, D
C	D
D	E
E	-

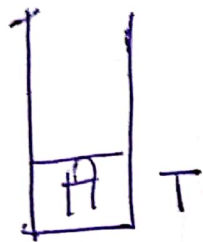
Status array

- 1- Initialize all node
- 2- Put $\downarrow$ in the stack node
3. Remove node in the stack

status array

A	B	C	D	E
1	1	1	1	1

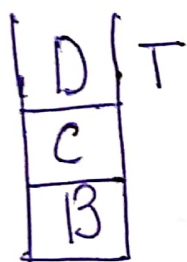
Push A into the stack



status array

A	B	C	D	E
2	1	1	1	1

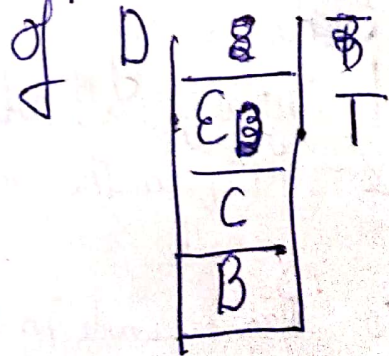
Pop and print A. Push all neighbouring node of A



Print A

A	B	C	D	E
3	2	2	2	1

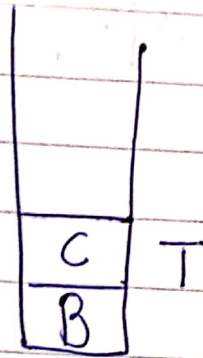
Pop and print D. Push all neighbouring node of D



A	B	C	D	E
3	2	2	3	2

Print D

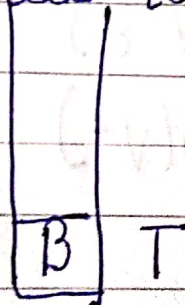
Pop and print E. Push all neighbouring node



A	B	C	D	E
3	2	2	3	3

Print E

Pop and print C. Push all neighbouring node of C. Adjacent of C is D which is already visit no need to add into the stack



A	B	C	D	E
3	2	3	3	3

Print C

Pop and print B. Push all the neighbouring node of B. Note that adjacent node of B is E, D which is already visit so no need to

add into the stack



A	B	C	D	E
3	3	3	3	3

Print B

Time Complexity

Using adjacency list = $O(V+E)$

" adjacency matrix = $O(V^2)$

Space Complexity

Using adjacency list = $O(V+E)$

" adjacency matrix = $O(V^2)$

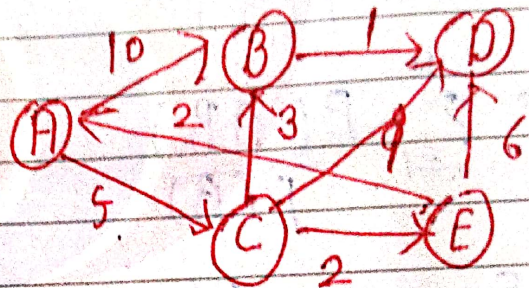
do for $j = 1$ to n
do $d_{ij}^{(k)} \leftarrow \min(d_{ij}^{(k-1)}, d_{ik}^{(k-1)} + d_{kj}^{(k-1)})$
return $D^{(n)}$

$\Theta(n^3)$

Sparse graph - adjacency list
dense graph - adjacency matrix

Dijkstra algorithm - Single Source Shortest algorithm

Source vertex = A



	A	B	C	D	E
A	0	∞	∞	∞	∞
C		10	5	∞	∞
E		8	14	7	
B	10	0	3	13	
D			4	0	

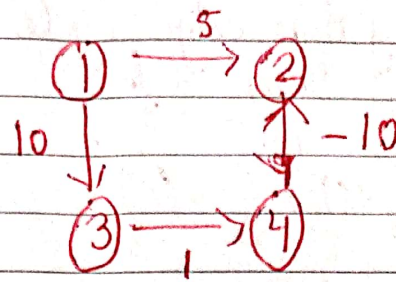
if $(d[u] + c[u, v] < d[v])$
 $d[v] = d[u] + c(u, v)$

Shortest Distance from A to D

A C B D

D B C A

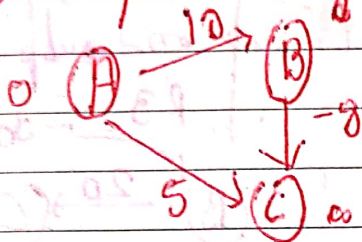
	1	2	3	4
1	0	∞	∞	∞
2	5	10	∞	
3			1	



2 is already been visited we cannot update the distance

Drawback:- It has negative edge it may or may not work.

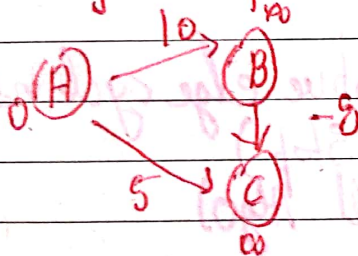
Bellmanford Algorithm.



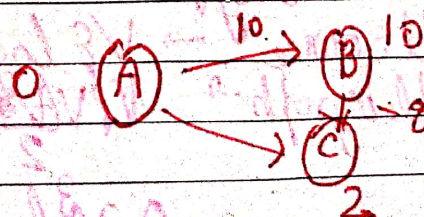
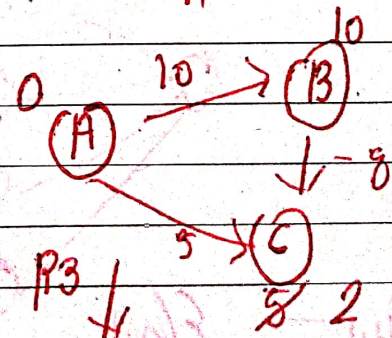
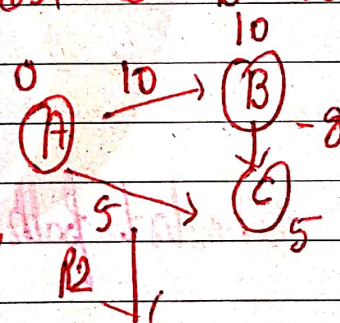
	A	B	C
A	0	∞	∞
C		10	5
B		2	X

Relax

Dijkstra fail to find minimum distance B $\rightarrow$ 10 C $\rightarrow$ 5.

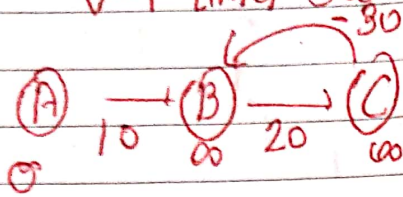


(V-1) times
Relax
R1

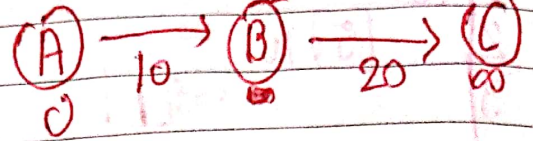




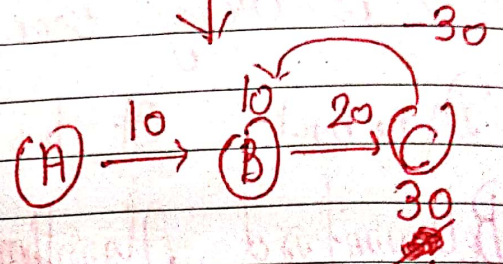
Negative edge cycle
V-9 times Bellman



R1

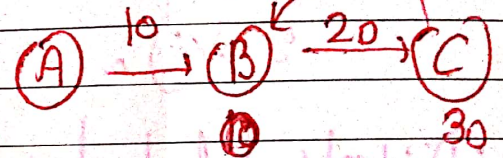


R2



(Stop and output)

R3

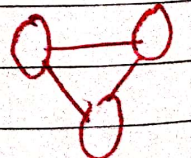
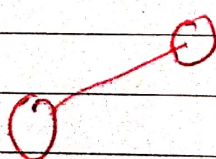
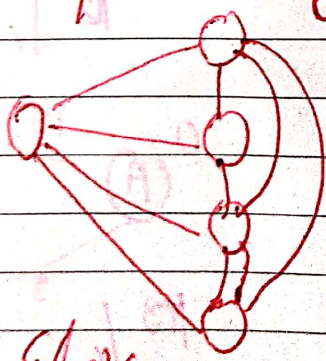


(Negative edge cycle and Stop)

+1 → check negative
stop cycle

It does not work with
Negative edge cycle (means
Sum is negative)
It works well with negative
edge cycle (means Sum is
positive)

All pairs Shortest path (Floyd Warshall Algo)



Dijkstra → $E \log V$

For all source → $V E \log V$

For complete graph → $V(V-1) \log V$

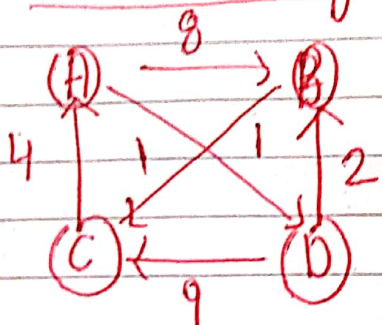
$\Theta V^3 \log V$

Bellman Ford Time complexity = VE
 (for single source)
 (for all pair) = V^2E

$$= V^2V^2 = O(V^4)$$

Use greedy approach $\rightarrow$ Dijkstra and Bellman Ford.

Floyd Warshall Algorithm



Distance matrix D^0
 (Direct Edge)

	A	B	C	D
A	0	8	∞	1
B	∞	0	1	∞
C	4	∞	0	∞
D	∞	2	9	0

Distance matrix (D^1)

	A	B	C	D
A	0	8	∞	1
B	∞	0	1	∞
C	4	12	0	5
D	∞	2	9	0

A-B

$$A-A + A-B$$

$$I) B-C = 1$$

$$B-A + A-C$$

$$II) B-D = \infty$$

$$B-A + A-D$$

$$III) C-B = \infty$$

$$C-A + A-B$$

$$4 + 8 = 12$$

$$C-D = \infty$$

$$C-A + A-D$$

$$4 + 1$$

A-D

$$A-B + B-D (B-C, C-D)$$

$$A-C = \infty$$

A-D

D-A

$$A-B + B-C$$

$$A-B + B-D$$

$$C-A$$

$$C-B + B-A$$

$$D-B + B-A$$

$$8 + 1 = 9$$

$$C-D \Rightarrow C-B + B-D$$



$$D_3 =$$

	A	B	C	D
A	0	8	9	1
B	5	0	1	6
C	4	12	0	5
D	7	2	3	0

B-D

B-C + C-D

(via A,

~~C-A~~ B)

C-A-B-D

A-B

A-C + C-B

B-A

B-C + C-A

1 + 4

B-D

B-C + C-D

1 + 4 + 8 + 1 = 6

$$D^4 =$$

	A	B	C	D
A	0	6	4	1
B	5	0	1	6
C	4	7	0	5
D	7	2	3	0

Final result.

Optimal cost Binary Search Key Tree

- Search time of an element in BST is $O(n)$.
 whereas in a Balanced Binary Search tree is $O(\log n)$. Again the Search time can be improved in optimal cost Binary Search tree.